

Đánh giá tính khả thi mô hình kinh doanh giao đồ ăn hướng shipper và cảm nang xây dựng giao diện khách hàng thử nghiệm

Sự bùng nổ của các nền tảng gọi món trực tuyến lớn tại Việt Nam đã thiết lập một hệ sinh thái kết nối chặt chẽ giữa nhà hàng, khách hàng và đội ngũ giao hàng. Tuy nhiên, cơ cấu phân bổ lợi nhuận hiện tại đang bộc lộ nhiều điểm nghẽn, đặc biệt là tỷ lệ chiết khấu cao đối với nhà hàng và mức thu nhập hạn chế trên mỗi đơn hàng của đối tác tài xế. Nhằm giải quyết vấn đề này, ý tưởng về một ứng dụng giao đồ ăn lấy tài xế làm trung tâm (Shipper-Centric Model) đã được đề xuất. Nghiên cứu này tập trung phân tích chuyên sâu tính khả thi của mô hình kinh doanh mới, đồng thời cung cấp mã nguồn giao diện khách hàng giả lập đạt độ chân thực tuyệt đối cùng khung kỹ thuật kiểm thử đồng bộ để đánh giá thực địa.

Phân tích tính khả thi của mô hình kinh doanh giao đồ ăn hướng shipper

Ý tưởng cốt lõi của mô hình đề xuất dựa trên việc tận dụng khoảng chênh lệch giữa giá bán tại quán và giá niêm yết trên ứng dụng hiện hành. Thay vì nộp chiết khấu từ 20% đến 30% cho nền tảng trung gian, nhà hàng sẽ bán đúng giá gốc tại quán, khách hàng trả đúng mức giá ứng dụng sẵn có, và toàn bộ phần chênh lệch này sẽ được chuyển trực tiếp vào thu nhập của tài xế giao hàng.

So sánh cơ cấu tài chính giữa hai mô hình vận hành

Để làm rõ bản chất kinh tế, bảng dưới đây so sánh chi tiết dòng tiền của một đơn hàng cụ thể (ví dụ: tô hủ tiếu có giá tại quán là 50.000 VNĐ và giá hiển thị trên ứng dụng ShopeeFood là 65.000 VNĐ):

Tham số tài chính	Mô hình ShopeeFood hiện hành	Mô hình đề xuất (Shipper-Centric)	Phân tích tác động và ý nghĩa kinh tế
Giá món tại quán	50.000 VNĐ	50.000 VNĐ	Mức giá gốc đảm bảo biên lợi nhuận tiêu chuẩn cho nhà hàng khi bán trực tiếp.
Giá bán trên ứng dụng	65.000 VNĐ	65.000 VNĐ	Khách hàng không phải trả thêm chi phí so với thói quen tiêu dùng hiện tại trên app lớn.
Chiết khấu từ nhà hàng	~20% đến 25% (13.000 VNĐ)	0% (0 VNĐ)	Nhà hàng thu về trọn vẹn 50.000 VNĐ, tăng tính hợp tác và loại bỏ

Tham số tài chính	Mô hình ShopeeFood hiện hành	Mô hình đề xuất (Shipper-Centric)	Phân tích tác động và ý nghĩa kinh tế
			áp lực tăng giá món trên ứng dụng.
Thu nhập của Shipper	9.000 VNĐ	15.000 VNĐ	Thu nhập tài xế tăng 66,7%, cải thiện đáng kể tỷ lệ giữ chân và động lực làm việc.
Doanh thu của nền tảng	~4.000 VNĐ (phí hệ thống + chiết khấu còn lại)	0 VNĐ (Chưa tính phí vận hành)	Điểm nghẽn lớn nhất: Nền tảng tự vận hành không có nguồn thu để duy trì hạ tầng kỹ thuật.
Cơ chế thanh toán	Điện tử / Ví ShopeePay / Tiền mặt	Shipper ứng vốn mua hộ (COD)	Đòi hỏi tài xế phải có sẵn dòng tiền lưu động lớn và chịu hoàn toàn rủi ro bùng đơn.

Đánh giá các chiều kích khả thi hệ thống (Feasibility Matrix)

Việc chuyển dịch từ ý tưởng sang một nền tảng công nghệ hoạt động ổn định đòi hỏi phải đánh giá khắt khe qua bốn khía cạnh cốt lõi:

- **Tính khả thi về mặt Tài chính:** Mô hình này mang lại lợi ích tuyệt đối cho tài xế và nhà hàng, nhưng lại đặt ra bài toán sinh tồn cho đơn vị vận hành ứng dụng. Nếu tài xế nhận trọn vẹn 15.000 VNĐ chênh lệch, nền tảng sẽ không có kinh phí để chi trả cho hệ thống máy chủ, lưu trữ cơ sở dữ liệu MongoDB, vận hành API NestJS, và đặc biệt là chi phí sử dụng API bản đồ (như Google Maps API) để định vị đơn hàng. Để khả thi, ứng dụng bắt buộc phải thu một khoản phí bản quyền cố định (ví dụ: 1.000 VNĐ đến 1.500 VNĐ trên mỗi đơn hàng hoàn thành) từ tài xế hoặc hiển thị quảng cáo địa phương.
- **Tính khả thi về mặt Vận hành:** Rào cản lớn nhất là việc duy trì tính chính xác của cơ sở dữ liệu giá món ăn. Giá tại quán (50.000 VNĐ) và giá trên ứng dụng đối thủ (65.000 VNĐ) liên tục biến động theo thời gian, chương trình khuyến mãi và chính sách của từng quán. Nếu không có sự hợp tác trực tiếp từ chủ quán để cập nhật biểu giá gốc, việc tài xế tự đến mua có thể phát sinh chênh lệch thực tế ngoài dự tính (ví dụ: quán tăng giá gốc lên 55.000 VNĐ), trực tiếp làm giảm thu nhập của shipper xuống còn 10.000 VNĐ và gây tranh chấp tài chính. Ngoài ra, cơ chế "mua hộ" buộc tài xế phải tự ứng trước tiền túi để trả cho quán, dẫn đến rủi ro mất trắng nếu khách hàng hủy đơn ở phút chót.
- **Tính khả thi về mặt Công nghệ:** Việc phát triển ứng dụng hoàn toàn nằm trong tầm tay của các đội ngũ lập trình hiện nay. Bằng cách sử dụng các framework mã nguồn mở phổ biến như React Native với Expo cho ứng dụng di động, kết hợp NestJS và MongoDB cho hệ thống Backend API, chi phí xây dựng ban đầu có thể được tối ưu hóa tối đa. Thử thách công nghệ nằm ở thuật toán ghép đơn thông minh (batching) và tối ưu hóa lộ trình di chuyển để một tài xế có thể giao nhiều đơn trên cùng một tuyến đường nhằm nhân đôi hiệu suất thu nhập.
- **Tính khả thi về mặt Pháp lý:** Tại Việt Nam, hoạt động sàn giao dịch thương mại điện tử và dịch vụ giao nhận vận chuyển bắt buộc phải đăng ký và phê duyệt bởi Bộ Công Thương. Đồng thời, các vấn đề về an toàn vệ sinh thực phẩm khi xảy ra sự cố ngộ độc độc lập, bảo hiểm tai nạn cho tài xế công nghệ, và nghĩa vụ khai thuế thu nhập cá nhân là những rào cản pháp lý phức tạp cần có pháp nhân doanh nghiệp đứng ra bảo trợ và chịu

trách nhiệm dân sự.

Giao diện khách hàng giả lập 100% phục vụ thử nghiệm thực địa

Để kiểm chứng phản ứng của người dùng và tính khả thi của quy trình giao nhận, một giao diện khách hàng mô phỏng độ phân giải cao (High-Fidelity) đã được xây dựng. Giao diện dưới đây sử dụng trực tiếp thư viện Tailwind CSS, tối ưu hóa hiển thị di động (Mobile-First Layout), mô phỏng hoàn chỉnh từng nút bấm, trạng thái giỏ hàng, và bảng phân rã chi phí minh bạch.

Mã nguồn HTML/Tailwind CSS giao diện di động hoàn chỉnh

Kiểm thử viên có thể sao chép toàn bộ đoạn mã dưới đây, lưu thành tệp .html và khởi chạy trực tiếp trên bất kỳ trình duyệt di động nào để tiến hành thử nghiệm thực địa với khách hàng và tài xế:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Giao Diện Kiểm Thử - Ứng Dụng Giao Đồ Ăn Hưởng
Shipper</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
  <style>
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe
UI", Roboto, Helvetica, Arial, sans-serif;
    }
  </style>
</head>
<body class="bg-gray-100 flex justify-center items-center
min-h-screen">

  <div class="w-full max-w-md bg-white min-h-screen shadow-2xl
relative flex flex-col overflow-hidden border border-gray-200">

    <div class="bg-red-600 text-white px-4 py-2 flex
justify-between items-center text-xs font-semibold z-10">
      <div>12:30</div>
      <div class="flex items-center space-x-2">
        <i class="fas fa-signal"></i>
        <i class="fas fa-wifi"></i>
        <i class="fas fa-battery-full"></i>
      </div>
    </div>
  </div>
</body>
```

```

        </div>
    </div>

    <div class="bg-red-500 text-white p-4 shadow-md z-10">
        <div class="flex items-center space-x-3">
            <button class="hover:opacity-80 transition-opacity"><i
class="fas fa-arrow-left text-lg"></i></button>
            <div class="flex-1">
                <p class="text-xs text-red-200">GIAO ĐẾN</p>
                <p class="text-sm font-bold truncate"><i
class="fas fa-map-marker-alt mr-1"></i> 123 Nguyễn Thị Minh Khai, Quận
1, TP. HCM</p>
            </div>
            <button class="relative hover:opacity-80
transition-opacity">
                <i class="fas fa-shopping-bag text-lg"></i>
                <span class="absolute -top-2 -right-2
bg-yellow-400 text-red-900 text-[10px] font-bold rounded-full w-4 h-4
flex items-center justify-center">1</span>
            </button>
        </div>
    </div>

    <div class="flex-1 overflow-y-auto pb-32">
        <div class="relative h-44 bg-gray-300">
            
            <div class="absolute inset-0 bg-gradient-to-t
from-black/70 to-transparent"></div>
            <div class="absolute bottom-3 left-4 text-white">
                <span class="bg-red-500 text-[10px] font-bold
uppercase px-2 py-0.5 rounded-full">Đối Tác Shipper</span>
                <h1 class="text-xl font-bold mt-1">Hủ Tiếu Nam
Vang - A Tài</h1>
                <p class="text-xs text-gray-200 flex items-center
mt-1">
                    <i class="fas fa-star text-yellow-400
mr-1"></i> 4.8 (120+ đánh giá) | 1.2 km | <i class="fas fa-motorcycle
ml-2 mr-1"></i> Giao nhanh 15-20 phút
                </p>
            </div>
        </div>
    </div>

    <div class="m-4 p-3 bg-green-50 border border-green-200
rounded-xl">
        <div class="flex items-start space-x-3">

```

```

        <i class="fas fa-info-circle text-green-600 mt-0.5
text-lg"></i>
        <div class="text-xs text-green-800 space-y-1">
            <p class="font-bold text-sm">Cam kết Minh Bạch
Giá Trị!</p>
            <p>Ứng dụng này hoạt động theo mô hình phi lợi
nhuận từ trung gian. Toàn bộ phí chênh lệch giá dịch vụ được chuyển
thẳng cho tài xế để hỗ trợ thu nhập thực tế của họ.</p>
        </div>
    </div>
</div>

<div class="px-4 py-2">
    <h2 class="text-sm font-bold text-gray-800 uppercase
tracking-wider border-b pb-2">Món ăn nổi bật</h2>

    <div class="flex py-4 border-b items-center
justify-between">
        <div class="flex-1 pr-4">
            <div class="flex items-center space-x-1.5">
                <span class="w-2.5 h-2.5 bg-red-600
rounded-full inline-block" title="Món mặn"></span>
                <h3 class="font-bold text-gray-800
text-base">Hủ Tiếu Nam Vang Đặc Biệt</h3>
            </div>
            <p class="text-xs text-gray-500 mt-1
line-clamp-2">Hủ tiếu dai ngon với tôm tươi, thịt băm, trứng cút và
nước lèo hầm xương đậm đà từ quán gốc.</p>
            <div class="flex items-center space-x-3 mt-2">
                <span class="text-red-600 font-extrabold
text-base">65.000đ</span>
                <span class="text-xs text-gray-400
line-through">Giá tại quán: 50.000đ</span>
            </div>
        </div>
        <div class="relative w-24 h-24 flex-shrink-0">
            
            <div class="absolute -bottom-2 left-1/2
transform -translate-x-1/2 bg-white border border-red-500 text-red-500
font-bold text-xs px-4 py-1 rounded-full shadow-md hover:bg-red-50
transition-colors cursor-pointer select-none">
                Đã Thêm
            </div>
        </div>
    </div>
</div>

```

```

        </div>
    </div>

    <div class="absolute bottom-0 left-0 right-0 bg-white border-t
border-gray-200 p-4 shadow-[0_-4px_10px_rgba(0,0,0,0.05)] z-20">
        <div class="mb-3 space-y-1 text-xs border-b pb-3">
            <div class="flex justify-between text-gray-600">
                <span>Tổng tiền món (Giá tại quán)</span>
                <span class="font-semibold">50.000đ</span>
            </div>
            <div class="flex justify-between text-green-600
font-medium">
                <span>Hỗ trợ thu nhập Shipper (Phần chênh lệch
món)</span>
                <span>+15.000đ</span>
            </div>
            <div class="flex justify-between text-gray-600">
                <span>Phí dịch vụ hệ thống</span>
                <span>0đ</span>
            </div>
            <div class="flex justify-between text-gray-800
font-bold text-sm pt-1">
                <span>Tổng chi phí thanh toán (COD)</span>
                <span class="text-red-600
text-base">65.000đ</span>
            </div>
        </div>
    </div>

    <div class="bg-yellow-50 border border-yellow-200 p-2
rounded-lg flex items-center space-x-2 mb-3">
        <i class="fas fa-id-card text-yellow-600 text-sm"></i>
        <p class="text-[11px] text-yellow-800 leading-tight">
            Tài xế nhận về tổng cộng <strong
class="text-red-600 text-xs">15.000đ</strong> cho đơn hàng này (thay
vì 9.000đ như trên các ứng dụng khác) [User Query].
        </p>
    </div>

    <button onclick="simulateOrderSuccess()" class="w-full
bg-red-500 hover:bg-red-600 active:scale-95 transition-transform
text-white font-bold py-3 px-4 rounded-xl shadow-lg flex
justify-between items-center text-sm">
        <span>XÁC NHẬN ĐẶT ĐƠN - TRẢ 65.000đ</span>
        <span class="bg-red-700 text-xs px-2 py-1
rounded">Giao Ngay <i class="fas fa-chevron-right ml-1"></i></span>
    </button>
</div>

```

```

        <div id="successModal" class="hidden absolute inset-0
bg-black/60 z-30 flex items-center justify-center p-6">
            <div class="bg-white rounded-2xl p-6 w-full max-w-sm
text-center shadow-2xl space-y-4">
                <div class="w-16 h-16 bg-green-100 rounded-full flex
items-center justify-center mx-auto text-green-500 text-3xl">
                    <i class="fas fa-check-circle"></i>
                </div>
                <div>
                    <h3 class="text-lg font-bold text-gray-800">Đặt
hàng thành công!</h3>
                    <p class="text-xs text-gray-500 mt-1">Đơn hàng của
bạn đã được chuyển đến hệ thống tài xế tự do lân cận.</p>
                </div>
                <div class="bg-gray-50 p-3 rounded-lg text-left
text-xs space-y-1.5 border">
                    <div class="flex justify-between">
                        <span class="text-gray-500">Mã đơn
hàng:</span>
                        <span class="font-mono font-bold
text-gray-700">SPF-882901</span>
                    </div>
                    <div class="flex justify-between">
                        <span class="text-gray-500">Tiền shipper trả
quán:</span>
                        <span class="font-bold text-gray-800">50.000
VNĐ</span>
                    </div>
                    <div class="flex justify-between">
                        <span class="text-gray-500">Khách trả
shipper:</span>
                        <span class="font-bold text-red-600">65.000
VNĐ</span>
                    </div>
                    <div class="flex justify-between border-t pt-1.5
mt-1">
                        <span class="font-bold text-green-700">Shipper
thực nhận:</span>
                        <span class="font-extrabold
text-green-700">15.000 VNĐ</span>
                    </div>
                </div>
                <button onclick="closeModal()" class="w-full
bg-gray-800 hover:bg-gray-900 text-white font-bold py-2.5 rounded-lg
text-xs transition-colors">
                    QUAY LẠI TRANG THỬ NGHIỆM
                </button>
            </div>

```

```

        </div>

</div>

<script>
    function simulateOrderSuccess() {

document.getElementById('successModal').classList.remove('hidden');
    }
    function closeModal() {

document.getElementById('successModal').classList.add('hidden');
    }
</script>
</body>
</html>

```

Giải pháp giả lập và kiểm thử API có trạng thái bằng MSW và Faker.js

Để hỗ trợ kiểm thử tự động quy trình vận hành và tính toán dữ liệu của ứng dụng, các kỹ sư phần mềm sử dụng giải pháp đánh chặn yêu cầu tại biên trình duyệt bằng thư viện Mock Service Worker (MSW) kết hợp bộ tạo dữ liệu chuẩn hóa Việt Nam của [@faker-js/faker](#) (fakerVI).

Kiến trúc giả lập luồng thanh toán và cập nhật giá món

Để mô phỏng hoàn chỉnh luồng nghiệp vụ đặc thù của ứng dụng (trong đó cần lưu giữ thông tin giá bán tại quán, giá hiển thị ứng dụng, và tính toán phần thu nhập thực nhận của tài xế), hệ thống API giả lập cục bộ phải lưu giữ trạng thái đơn hàng (Stateful Mocking).

```

import { http, HttpResponse, delay } from 'msw';
import { fakerVI } from '@faker-js/faker';

// Khai báo cấu trúc dữ liệu mô phỏng nghiệp vụ thực tế của đơn hàng
interface MerchantItem {
  itemId: string;
  foodName: string;
  inStorePrice: number;    // Giá bán trực tiếp tại quán (ví dụ:
50.000 VNĐ)
  appPrice: number;        // Giá niêm yết trên ứng dụng khách (ví
dụ: 65.000 VNĐ)
}

interface SimulatedOrder {
  orderId: string;
  restaurantName: string;

```



```

    shipperId: string;
    shipperName: string;
    items: Array<{
        item: MerchantItem;
        quantity: number;
    }>;
    cashPaidToStore: number; // Số tiền mặt shipper phải trả cho quán
    ăn
    cashCollectedFromUser: number; // Số tiền mặt khách hàng trả cho
    shipper
    shipperRevenue: number; // Thu nhập thực nhận của shipper từ đơn
    hàng
    status: 'PENDING' | 'ACCEPTED' | 'PURCHASED' | 'DELIVERED' |
    'FAILED';
}

// Khởi tạo cơ sở dữ liệu giả lập trong bộ nhớ (In-Memory Database)
const activeOrders: Map<string, SimulatedOrder> = new Map();

export const deliveryAppHandlers = [
    // 1. Lấy danh sách món ăn từ nhà hàng lân cận kèm bảng phân rã giá
    http.get('/api/v1/restaurants/:id/menu', async () => {
        await delay(200); // Mô phỏng độ trễ truyền dẫn mạng
[span_49](start_span)[span_49](end_span)
        const mockMenu: MerchantItem =;
        return HttpResponse.json(mockMenu);
    }),

    // 2. Tiếp nhận yêu cầu khởi tạo đơn hàng mới và tính toán cơ cấu
    đồng tiền
    http.post('/api/v1/orders/create', async ({ request }) => {
        const requestData = (await request.json()) as {
            restaurantName: string;
            items: Array<{ itemId: string; quantity: number }>;
        };

        const generatedOrderId = `SPF-${fakerVI.number.int({ min: 100000,
max: 999999 })}`;

        // Thiết lập các thông số tài chính cho đơn hàng dựa trên nghiệp
        vụ
        let cashPaidToStore = 0;
        let cashCollectedFromUser = 0;

        // Bản đồ dữ liệu thực đơn cứng để đối chiếu tính toán
        const menuLookup: Record<string, MerchantItem> = {
            'item-001': { itemId: 'item-001', foodName: 'Hủ Tiếu Nam Vang
Đặc Biệt', inStorePrice: 50000, appPrice: 65000 },

```

```

        'item-002': { itemId: 'item-002', foodName: 'Mì Hoàn Thánh Xá
Xíu', inStorePrice: 45000, appPrice: 58000 },
    };

    const resolvedItems = requestData.items.map(cartItem => {
        const menuDetail = menuLookup[cartItem.itemId];
        cashPaidToStore += menuDetail.inStorePrice * cartItem.quantity;
        cashCollectedFromUser += menuDetail.appPrice *
cartItem.quantity;
        return {
            item: menuDetail,
            quantity: cartItem.quantity
        };
    });

    const shipperRevenue = cashCollectedFromUser - cashPaidToStore;

    const newOrder: SimulatedOrder = {
        orderId: generatedOrderId,
        restaurantName: requestData.restaurantName,
        shipperId: `SHIP-${fakerVI.number.int({ min: 1000, max: 9999
    })}`,
        shipperName: fakerVI.person.fullName(),
        items: resolvedItems,
        cashPaidToStore,
        cashCollectedFromUser,
        shipperRevenue,
        status: 'PENDING',
    };

    // Lưu trữ trạng thái đơn hàng phục vụ các bước truy vấn trạng
thái tiếp theo [span_50] (start_span) [span_50] (end_span)
    activeOrders.set(generatedOrderId, newOrder);

    return HttpResponse.json(newOrder, { status: 201 });
}),

// 3. Tài xế thay đổi trạng thái đơn hàng trong quá trình vận chuyển
thực tế
    http.patch('/api/v1/orders/:orderId/status', async ({ params,
request }) => {
        const { orderId } = params;
        const { newStatus } = (await request.json()) as { newStatus:
SimulatedOrder['status'] };

        const existingOrder = activeOrders.get(orderId as string);
        if (!existingOrder) {
            return HttpResponse.json({ error: 'Đơn hàng không tồn tại' }, {

```

```

status: 404 });
    }

    existingOrder.status = newStatus;
    activeOrders.set(orderId as string, existingOrder);

    return HttpResponseMessage.json(existingOrder);
  })
];

```

Danh mục kịch bản kiểm thử tích hợp hệ thống (Integration Test Suite)

Để giao diện giả lập và hệ thống dịch vụ vận hành chính xác dưới các tác động ngoại cảnh hoặc ngắt kết nối đột ngột, quy trình kiểm thử hệ thống cần phủ kín các trường hợp kiểm thử sau :

Phân nhóm kiểm thử	Mã kịch bản	Mô tả chi tiết hành động tương tác	Trạng thái mong đợi (Expected Behavior)
Xử lý bất đồng bộ	IT-ASYNC-01	Người dùng ấn xác nhận thanh toán khi mạng kết nối chậm chạp (mô phỏng mất mạng 3G/4G giữa chừng).	Ứng dụng kích hoạt bộ đếm thời gian chờ (Timeout) tối đa 100 giây; hiển thị thông báo tiến trình đang được đồng bộ thay vì báo lỗi trực tiếp.
Khớp giá đơn hàng	IT-CALC-01	Người dùng đặt mua 2 tô hủ tiếu trong giỏ hàng và tiến hành kiểm tra hóa đơn tính toán thanh toán.	Tổng tiền mặt khách hàng chuẩn bị trả là 130.000 VNĐ; tiền shipper thanh toán cho quán ăn là 100.000 VNĐ; thu nhập của shipper được ghi nhận chính xác 30.000 VNĐ.
Tích hợp định vị	IT-GEO-01	Thay đổi địa chỉ nhận hàng tại màn hình thanh toán từ khu vực Quận 1 sang Quận 12.	Hệ thống định vị gọi API ước tính quãng đường mới, tự động phát hiện nếu khoảng cách vượt quá 10km và từ chối tiếp nhận đơn hàng do vượt ngưỡng phục vụ của shipper tự do.
Bảo mật giao thức	IT-SEC-01	Gửi yêu cầu cập nhật trạng thái đơn hàng mà không đính kèm mã xác thực Token hợp lệ	Hệ thống Backend trả về mã lỗi 401 Unauthorized; ngăn chặn hành vi can thiệp

Phân nhóm kiểm thử	Mã kịch bản	Mô tả chi tiết hành động tương tác	Trạng thái mong đợi (Expected Behavior)
		trên Header.	trái phép vào tiến trình giao hàng.

Hướng dẫn triển khai tích hợp luồng xử lý và vận hành thử nghiệm

Để đưa giao diện khách hàng và hệ thống giả lập vào quá trình thử nghiệm thực địa, nhóm phát triển cần thực hiện tuần tự các bước cấu hình kỹ thuật dưới đây nhằm đảm bảo tính đồng bộ :

1. **Khởi tạo môi trường phát triển cục bộ:** Cài đặt môi trường Node.js phiên bản tối thiểu v18 trở lên trên máy tính kiểm thử. Khởi tạo dự án và tiến hành cài đặt các gói thư viện cần thiết phục vụ cho việc giả lập dữ liệu:

```
npm init -y
npm install msw @faker-js/faker --save-dev
```

2. **Thiết lập thư mục dự án và tích hợp Service Worker:** Tạo tệp tin lưu trữ các đoạn mã cấu hình API giả lập trong thư mục dự án. Tiếp theo, khởi chạy lệnh tích hợp của thư viện MSW vào thư mục công cộng của ứng dụng web để đăng ký trình chạy ngầm (Service Worker) ở tầng trình duyệt :

```
npx msw init public/ --save
```

3. **Cấu hình khởi chạy kiểm thử thực địa:** Nhúng đoạn mã kích hoạt trình giả lập vào tệp tin khởi chạy ứng dụng chính (như index.js hoặc main.ts) của giao diện khách hàng. Cơ chế này đảm bảo rằng hệ thống chỉ kích hoạt lớp giao tiếp giả lập khi ứng dụng chạy trong môi trường phát triển hoặc thử nghiệm cục bộ :

```
async function enableMocking() {
  if (process.env.NODE_ENV !== 'development') {
    return;
  }
  const { worker } = await import('./mocks/browser');
  return worker.start({
    onUnhandledRequest: 'bypass', // Bỏ qua không đánh chặn các yêu cầu tải tài nguyên tĩnh bên ngoài
  });
}

enableMocking().then(() => {
  // Khởi chạy ứng dụng giao diện khách hàng sau khi MSW đã hoạt động ổn định
  mountApp();
});
```

Việc triển khai đồng bộ giải pháp kiểm thử giao diện thực tế cùng hệ thống giả lập dữ liệu động được bản địa hóa giúp nhóm phát triển nhanh chóng thu thập các số liệu phản hồi thực tế từ thị trường. Từ đó, doanh nghiệp có thể đưa ra các điều chỉnh tối ưu về cơ cấu chi phí, tối ưu hóa trải nghiệm tài xế và giảm thiểu tối đa các rủi ro vận hành trước khi tiến hành xây dựng và phát

hành phiên bản chính thức trên các kho ứng dụng di động.

Nguồn trích dẫn

1. Food & Grocery Delivery Apps: Testing Checklist - QATestLab Blog, <https://blog.qatestlab.com/food-grocery-delivery-apps-testing-checklist/> 2. GitHub - nvminh162/shopeefood-app: Project: Clone ShopeeFood with React Native (Cross platform), <https://github.com/nvminh162/shopeefood-app> 3. QA Testing Challenges for Tech Startups Explained - QAble, <https://www.qable.io/blog/testing-online-food-delivery-app> 4. Checkout Test | Importance, Test Cases & How to Test? - Testsigma, <https://testsigma.com/blog/checkout-test/> 5. Fast Food And Restaurant React Tailwind Template - Envato, <https://elements.envato.com/fast-food-and-restaurant-react-tailwind-template-J543JX3> 6. How We Cut Development Time by 60% Using Mock Service Worker ..., <https://medium.com/@hrupanjan/how-we-cut-development-time-by-60-using-mock-service-worker-faker-js-1e867b2498dc> 7. How to Test Food Delivery Apps: Key Test Cases & Best Practices - TestGrid, <https://testgrid.io/blog/testing-food-delivery-apps/> 8. Fast Food App - Free Delivery App UI Kit - UIBundle, <https://uibundle.com/products/1583-fast-food-app-free-delivery-app-ui-kit> 9. Restfast - Fast Food And Restaurant Tailwind CSS Template by rowDevs - ThemeForest, <https://themeforest.net/item/restfast-fast-food-and-restaurant-tailwind-css-template/58856162> 10. Supercharge Frontend Development with MSW, OpenAPI, and AI-Generated Mocks, <https://dev.to/michaliskout/supercharge-frontend-development-with-msw-openapi-and-ai-generated-mocks-1bfo> 11. Vietnamese - Faker.js, <https://fakerjs.dev/locales/vi.html> 12. msw-auto-mock - NPM, <https://www.npmjs.com/package/msw-auto-mock> 13. Food Delivery App Usability Testing Template - UXArmy, <https://uxarmy.com/templates/food-delivery-app-usability-testing-template/>